

Tutorial MARS (MIPS Assembler and Runtime Simulator)

Este tutorial proporciona una guía básica sobre cómo utilizar el simulador MARS para programar, ensamblar y depurar aplicaciones en lenguaje ensamblador MIPS.

1. Introducción a MARS

MARS es un entorno de desarrollo integrado (IDE) ligero para el lenguaje ensamblador MIPS. Es ampliamente utilizado en la academia debido a su simplicidad y a las potentes herramientas de visualización que ofrece.

Requisitos

- Tener instalado el **Java Runtime Environment (JRE)** versión 1.5 o superior.
 - Descargar el archivo **.jar** de MARS desde su sitio oficial.
-

2. Interfaz del Simulador

La interfaz de MARS se divide en tres áreas principales:

1. **Edit Tab (Editor):** Donde escribes el código fuente (**.asm**).
 2. **Execute Tab (Ejecución):** Se activa tras ensamblar el código. Permite ver la memoria, los registros y el código máquina.
 3. **Messages/Console:** En la parte inferior, muestra errores de compilación y la salida del programa (syscalls).
-

3. Cómo Programar en Assembler MIPS con MARS

Estructura de un archivo **.asm**

Un programa en MIPS generalmente tiene dos secciones:

```
.data
    # Aquí se definen las variables (datos en memoria)
    mensaje: .asciiz "Hola Mundo"
    numero:  .word 42

.text
.globl main
main:
    # Aquí va el código (instrucciones)
    li $v0, 4          # Cargar código de servicio: imprimir string
    la $a0, mensaje    # Cargar dirección del mensaje
    syscall            # Llamada al sistema
```

```
li $v0, 10          # Código para finalizar programa
syscall
```

Pasos para ejecutar un programa:

1. **Escribir:** Crea un nuevo archivo (**File** -> **New**) y escribe tu código.
2. **Guardar:** Guarda el archivo con extensión **.asm**.
3. **Ensamblar:** Haz clic en el icono de la **llave inglesa** (o pulsa **F3**). Si hay errores, aparecerán en la consola inferior.
4. **Ejecutar:**
 - Usa el icono del **Play verde** para ejecutar todo el programa.
 - Usa el icono de la **flecha con el 1** para ejecutar instrucción por instrucción (Paso a paso).
5. **Resetear:** Usa el icono de la **flecha azul circular** para reiniciar los registros y la memoria antes de volver a ejecutar.

4. Herramientas de Depuración (Debugging)

Visualización de Registros

En el panel derecho (o inferior, según la configuración), verás la pestaña **Registers**.

- Puedes modificar los valores de los registros manualmente haciendo doble clic sobre ellos.
- Los registros que cambian en la última instrucción ejecutada se resaltan en color verde.

Inspección de Memoria (Data Segment)

En la pestaña **Execute**, puedes ver el **Data Segment**.

- Puedes navegar entre la memoria de datos, el stack y el segmento de texto.
- Permite ver los valores en formato Hexadecimal o Decimal (desmarcando "Hexadecimal Values" en la parte inferior).

5. El Simulador de Pipeline y Herramientas de Análisis

Dependiendo de la versión de MARS, puedes usar diferentes herramientas para analizar el rendimiento:

1. **MIPS X-Ray:** Ve al menú **Tools** -> **MIPS X-Ray**. Esta herramienta muestra una animación del camino de datos (Datapath). Es excelente para ver cómo las instrucciones activan la ALU, la Memoria y los Registros.
2. **Instruction Statistics:** Ve a **Tools** -> **Instruction Statistics**. Permite ver cuántas instrucciones de cada tipo (Carga, Aritmética, Salto) se han ejecutado.
3. **Análisis de Riesgos:** Si tu versión de MARS no incluye el simulador de pipeline gráfico, el análisis de riesgos se realiza identificando manualmente las dependencias (como un **mul** que usa un registro cargado por un **lw** inmediatamente anterior) y verificando la mejora en el conteo total de instrucciones o ciclos.

6. Ejemplos Comunes de Código

Leer un número entero y mostrarlo

```
.text
main:
    li $v0, 5          # Servicio 5: Leer entero
    syscall
    move $t0, $v0      # Guardar el número leído en $t0

    li $v0, 1          # Servicio 1: Imprimir entero
    move $a0, $t0      # El número a imprimir debe estar en $a0
    syscall

    li $v0, 10         # Salir
    syscall
```

Bucle Simple (Contar de 0 a 9)

```
.text
main:
    li $t0, 0          # i = 0
    li $t1, 10         # Límite = 10

loop:
    beq $t0, $t1, fin  # Si i == 10, ir a fin

    # (Aquí iría el cuerpo del bucle)

    addi $t0, $t0, 1   # i++
    j loop

fin:
    li $v0, 10
    syscall
```

7. Atajos de Teclado Útiles

- **F3**: Ensamblar.
- **F5**: Ejecutar programa completo.
- **F7**: Ejecutar un paso (Step).
- **F8**: Ejecutar un paso hacia atrás (si el simulador lo permite).
- **Ctrl + Z**: Deshacer en el editor.